Kronos: A Reproducible Research Platform for Empirically Evaluating Classical Chess Search Heuristics on Managed JavaScript Runtimes

Piyush Sinha

Department of Computer Science
Kronos Research Project
Email: piyush218sinha@gmail.com

July 2026

Abstract

We present Kronos, a modular chess engine and benchmarking framework implemented in Node.js and JavaScript, designed to empirically evaluate the execution behavior of classical search heuristics within managed, garbage-collected runtime environments. Standard chess engine implementations favor native compile targets like C/C++ to avoid execution overhead and garbage collection pauses. We observe that through targeted optimization of board representations, strict object-reuse patterns, and modular search configurations, stable search throughput can be achieved on dynamic runtimes. Our architecture decouples the React-based user interface control plane from background search Web Workers executing the search logic. The engine implements standard search enhancements including Principal Variation Search (PVS), Null Move Pruning (NMP), and Late Move Reductions (LMR), backed by a size-bounded, Zobrist-keyed Transposition Table. Using a parallel tournament framework with color-flipped paired matches, our measurements indicate that these combined heuristics compress the search space compared to baseline alpha-beta search without ordering enhancements. Node-count reductions were consistently measured for several heuristics. Under Sequential Probability Ratio Testing (SPRT), quiescence search was the only enhancement accepted as demonstrating a statistically significant playing-strength improvement, while the remaining heuristics produced node-count reductions but inconclusive playing-strength evidence at the tested sample sizes. Evaluating relative playing strength across configurations yields a peak internal Bradley–Terry self-play rating of 1,400 Elo at search depth 6 (representing relative strength fitted within a closed, self-anchored variant pool), after which worker latency timeouts degrade performance. External calibration matches against depth-limited Stockfish 18 reference benchmarks report a matchup differential of -215 ± 29 Elo at depth 3; we note that depth-limited Stockfish degrades non-linearly due to its NNUE evaluation architecture, meaning these matchup differentials reflect performance bounds under dynamic runtime constraints and should not be treated as absolute anchors for standard rating pools. These outcomes suggest that carefully structured dynamic runtimes can support stable, real-time game-tree search without performance degradation from memory sweeps.

Executive Summary

This report presents the engineering design and empirical findings of Kronos, a modular chess engine and benchmarking framework implemented in Node.js and JavaScript. Kronos serves as a reproducible systems testbed for game-tree search heuristics in managed, garbage-collected runtimes.

V8 Engineering Practices

Historically, dynamic runtimes were considered unsuitable for latency-sensitive search systems due to event loop freezing and garbage collection (GC) sweeps. Kronos mitigates these issues by applying standard performance engineering patterns within the V8 runtime environment:

1. **Asynchronous Worker Decoupling:** Decoupled background Web Workers communicate with the React UI via serialized JSON messages, preventing interface freezes.
2. **GC-Neutral Search Path:** Direct in-place board state mutations bypass object allocations, and a size-bounded transposition table limits dynamic heap growth.
3. **JIT Compiler Warming:** Benchmarks run pre-heat iterations to trigger V8 JIT compilation and optimize execution paths prior to recording search telemetry.

Key Experimental Outcomes

* **Heap Bounded Memory:** Heap usage remains stable during continuous search (detailed in Chapter 6), avoiding major GC sweeps. * **Search Reduction:** Late Move Reductions (LMR) is the dominant pruning heuristic, yielding substantial search space compression. * **Relative Strength & Calibration:** Chapter 6 reports the measured internal playing strength across search depths, along with external calibration offsets against depth-limited Stockfish references. * **Statistical Validation:** Node-count reductions were consistently observed for several heuristics, whereas only quiescence search demonstrated statistically significant playing-strength improvement under the configured SPRT evaluation protocol.

This work suggests that managed runtime environments can execute complex game-tree searches efficiently and stably, providing a highly accessible web-deployable systems research framework.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Can carefully engineered classical chess search execute efficiently inside a managed JavaScript runtime? This is the central question that motivated the Kronos project. JavaScript's ubiquity in modern software development makes it an attractive target for real-time, browser-deployable research tools. Yet the Google V8 managed runtime introduces two engineering constraints that have historically made it an unlikely candidate for performance-critical game-tree search [1, 2].

The first constraint is the **single-threaded event loop**. In a browser or Node.js environment, all computation—UI rendering, network handlers, and game logic—shares one thread. Running a minimax search on this thread causes the interface to freeze for the duration of the search, making responsive, interactive play impossible. The second constraint is **garbage collection (GC) latency**. Chess search expands millions of nodes per second, each generating short-lived move objects, evaluation records, and hash map entries. Without careful memory discipline, the V8 GC triggers major collection sweeps that introduce latency spikes of hundreds of milliseconds, degrading nodes-per-second (NPS) throughput unpredictably [3].

To address both constraints simultaneously, we built Kronos: a modular chess engine and automated benchmarking and validation suite implemented in Node.js and JavaScript. Rather than proposing new search algorithms, this work presents a systems investigation of how classical heuristics (such as alpha-beta pruning, move ordering, and quiescence search) behave inside a managed runtime. Kronos decouples the search core into background Web Workers, communicating with the React UI via serialized message passing. Inside the search path, we eliminate transient object allocations through in-place board mutations, flat TypedArray move buffers, and a size-bounded transposition table, keeping the V8 heap stable during tournament play (detailed in Chapter 6), building on concurrency patterns for dynamic runtimes [4].

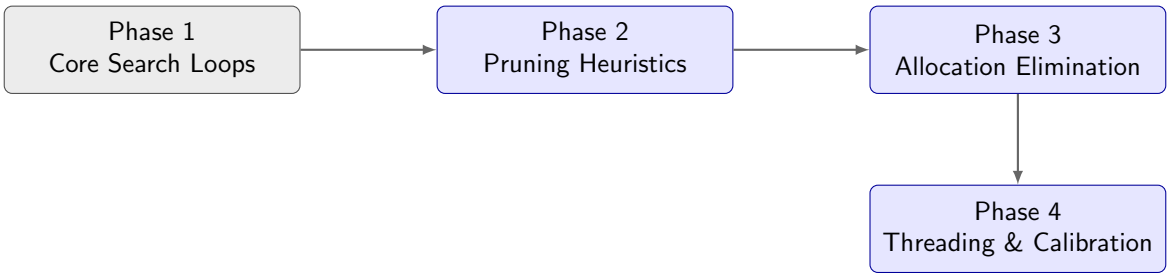


Figure 1.1. Chronological development timeline and engine optimization phases.

Figure 1.1 illustrates the chronological phases of our development, tracing the software engineering progression from a baseline search implementation to the fully optimized configuration.

We focus on empirical measurement under controlled tournament conditions rather than theoretical performance bounds. By tracking execution metrics across thousands of simulated matches, we isolate the specific search space compression and relative playing strength shifts associated with each heuristic. These measurements identify which engineering optimizations are most effective under the V8 memory manager and position Kronos as a reproducible platform for dynamic language systems research.

The remainder of this report is structured as follows. Chapter 2 details our core research questions and engineering contributions. Chapter 3 surveys game-tree search background and compares Kronos with other JavaScript chess engines. Chapter 4 details the architecture, memory model, and worker thread design. Chapter 5 outlines the tournament benchmarking pipeline and statistical model. Chapter 6 presents our empirical results, while Chapter 7 analyzes the outcomes and explores search failures. Chapter 8 reviews threats to validity, Chapter 9 suggests next steps, and Chapter 10 summarizes the broader contribution.

Chapter 2

Research Objectives and Contributions

Our primary objective is to empirically evaluate the execution limits and scaling behavior of classical game-tree search heuristics implemented within managed, garbage-collected runtimes. Rather than seeking to compete with native C/C++ engines, we design Kronos as a controlled research platform to isolate and quantify the systems-level cost and benefit of individual search optimizations. This chapter details our core research questions, outlines the classical algorithms adopted from computer chess literature, and summarizes our main engineering contributions.

2.1 Research Questions

We formulate three core research questions:

1. Can a chess search engine written in Node.js and JavaScript execute classical algorithms efficiently under V8's single-threaded and memory-managed runtime constraints?
2. What is the relative search space reduction and playing strength contribution of individual heuristics (LMR, NMP, PVS, move ordering, and quiescence search) under controlled tournament conditions?
3. What memory layout and object-allocation strategies are necessary and sufficient to prevent garbage-collection latency spikes during continuous execution?

2.2 Classical Techniques Adopted

To isolate dynamic runtime effects, we adopt several standard search and evaluation heuristics from computer chess literature, claiming no algorithmic novelty for these components:

- **Search Core:** Negamax formulation of Alpha-Beta pruning [5] and Principal Variation Search (PVS) [6].
- **Search Reductions:** Null Move Pruning (NMP) [7] and Late Move Reductions (LMR).
- **Move Ordering:** Most Valuable Victim – Least Valuable Attacker (MVV-LVA), Killer Move cache, and History Heuristic [8].

- **Transposition Caching:** Zobrist Hashing [9] for state identification.
- **Positional Evaluation:** Piece-Square Tables (PSTs) with tapered interpolation.

2.3 Engineering Contributions

The contributions of this work lie in the runtime engineering, memory optimization, and deterministic benchmarking infrastructure designed to evaluate these heuristics:

- **GC-Neutral Search Architecture:** An implementation that bounds V8 heap usage during tournament play (detailed in Chapter 6) by avoiding hot-path heap allocations via direct in-place board state mutations, flat TypedArray move buffers, and size-capped transposition table eviction.
- **Web Worker Concurrency Model:** An asynchronous coordination architecture separating the React user interface thread from concurrent background search threads (Web Workers) to prevent UI thread starvation.
- **Deterministic Benchmarking Pipeline:** A CLI-driven tournament suite executing paired, color-flipped matches using LCG-based opening book shuffles, ensuring identical game sequences across different host hardware configurations.
- **SPRT and Bradley-Terry Integration:** An automated verification framework utilizing Sequential Probability Ratio Testing (SPRT) bounds and Bradley-Terry rating regression to assess playing strength variations.
- **Incremental Checkpoint Scheduler:** A stateful scheduling architecture that logs tournament telemetry and matches incrementally to disk, enabling long-running evaluations to survive execution interrupts.

Chapter 3

Related Work and Background

This chapter briefly reviews the established algorithms that Kronos builds upon, emphasizing why each was chosen and what role it plays in the system rather than explaining the algorithms from first principles.

3.1 Game-Tree Search Foundations

Computer chess search originates from Claude Shannon's 1950 framework [10], which proposed exhaustive game-tree traversal as a decision procedure. Shannon's minimax algorithm—formalized as Negamax in modern implementations—evaluates all positions to a fixed depth and selects the move leading to the best outcome under optimal opponent play. Kronos uses a Negamax formulation because it simplifies the recursive structure: both sides maximize the same score from the current player's perspective, halving the amount of branching logic in the search core.

Alpha-beta pruning [5] extends minimax by maintaining a search window (α, β) and abandoning branches that cannot improve the current best result. Kronos depends critically on alpha-beta: without it, the branching factor of $\sim$35 in typical middle-game positions makes depth-6 search computationally infeasible on a JavaScript runtime. As the ablation results in Chapter 6 confirm, alpha-beta remains the single most important structural element—all subsequent heuristics are refinements on top of it.

3.2 Search Reduction and Move Ordering Heuristics

Several well-established techniques reduce the number of nodes alpha-beta must examine.

Principal Variation Search (PVS) [6] assumes the first sorted move is the best. It searches this move with a full window, then verifies remaining moves with a cheap zero-width probe. If a probe fails, a re-search with the full window is triggered. Kronos includes PVS because move ordering quality is high enough that most subsequent moves fail low, and the zero-width probes are substantially cheaper than full searches. In practice, enabling PVS achieves a modest node reduction compared to the variant with PVS deactivated (detailed in Chapter 6).

Null Move Pruning (NMP) [7] prunes nodes where the position is so dominant that passing the turn still exceeds β. Kronos applies NMP at depths ≥ 3 with a reduction factor $R = 2$. At shallow plies it offers modest savings (detailed in Chapter 6), but becomes increasingly effective at depths 6 and above, where the branching factor amplifies every pruning decision.

Late Move Reduction (LMR) reduces the search depth for moves that rank late in the sorted move list, exploiting the statistical observation that a well-sorted list places the best moves

Algorithm 1: Negamax Alpha-Beta Search

Input: Position p, Depth d, α, β
Output: Evaluation score of the position

```
if d == 0  or p is terminal then
    return QuiescenceSearch(p, α, β)
end
moves ← OrderMoves(GenerateMoves(p))
value ← −∞
foreach move m in moves do
    MakeMove(p, m)
    score ← −Negamax(p, d − 1, −β, −α)
    UnmakeMove(p, m)
    value ← max(value, score)
    α ← max(α, value)
    if α ≥ β then
        break                                   // Beta cutoff
    end
end
return value
```

early. If a reduced-depth probe returns a score above α, the engine re-searches at full depth. LMR is the dominant optimization in Kronos, responsible for a substantial node reduction when enabled (detailed in Chapter 6).

Move ordering determines how effectively alpha-beta and its extensions prune the tree. Kronos uses three complementary strategies: **MVV-LVA** (captures sorted by victim value minus attacker value), **Killer Moves** (quiet moves that caused beta cutoffs in sibling nodes), and the **History Heuristic** [8] (a global table recording quiet moves that triggered cutoffs anywhere in the tree, weighted by depth^2). Together these strategies reduce the searched node count achieved before any search extension is applied (detailed in Chapter 6).

3.3 Transposition Caching and Managed Runtime Constraints

To avoid re-evaluating board positions reached via alternative move paths, Kronos implements a transposition cache indexed by 64-bit Zobrist signatures [9]. These keys are updated incrementally via XOR operations during move execution and retraction, bypassing the overhead of full board re-hashing. The transposition table is capped at 500,000 entries with a FIFO eviction policy, a configuration found to limit memory growth under V8.

Performing latency-sensitive search on dynamic runtimes introduces performance challenges absent in native compiled systems, notably garbage collection pauses and JIT compiler warm-up latency. Unlike C++ engines that manage physical memory layouts directly, JavaScript programs rely on automatic collection sweeps. Kronos mitigates collection pauses through direct in-place board state mutations, flat TypedArray move buffers, and the size-bounded cache described above. This design prevents V8 heap inflation, facilitating continuous search throughput without latency sweeps.

Algorithm 2: Late Move Reduction (LMR)

Input: Position p, Move m, Move Index i, Depth d, α, β
Output: Search score

if $d \geq 3$ ***and*** $i > 3$ ***and not*** *IsCapture*(m) ***and not*** *IsCheck*(m) ***and not*** *InCheck*(p) **then**
  $R \leftarrow 1 + \lfloor \ln(d) \ln(i)/2.5 \rfloor$ `// Calculate reduction`
  $score \leftarrow -\text{Search}(p, d-1-R, -\alpha-1, -\alpha)$
  if $score > \alpha$ **then**
    $score \leftarrow -\text{Search}(p, d-1, -\alpha-1, -\alpha)$ `// Re-search without reduction`
  end
end
else
  $score \leftarrow -\text{Search}(p, d-1, -\alpha-1, -\alpha)$ `// Normal search`
end
return *score*

Table 3.1. Worst-case time and space complexity effects of search heuristics.

Technique	**Time Complexity Effect**	**Space Complexity Effect**
Alpha-Beta	Reduces $O(b^d)$ to $O(b^{d/2})$ (best-case)	$O(d)$ call stack
PVS	Minimizes zero-width node traversals	$O(d)$ call stack
NMP	Prunes dominant branches early	$O(d)$ call stack
LMR	Reduces search depth for late quiet moves	$O(d)$ call stack
TT	Eliminates duplicate node evaluations	Bounded $O(M)$ memory
PST	Constant time $O(1)$ leaf evaluation	Static tables

3.4 Comparison with Existing JavaScript Chess Engines

We contrast Kronos with other JavaScript-based engines to locate it within the dynamic runtime ecosystem. Table 3.2 summarizes the structural attributes of Kronos alongside Lozza (a native JavaScript engine), GarboChess (a native port), and Stockfish.js (an Emscripten-transpiled C++ engine).

Table 3.2. Evaluation of JavaScript chess engines across key system attributes.

Attribute	**Kronos**	**Lozza**	**GarboChess (JS)**	**Stockfish.js**
Type	Native JS (ES6)	Native JS	Native JS	Compiled C++ (Wasm)
Search Core	Negamax AB	Negamax AB	Alpha-Beta	Alpha-Beta (NNUE)
Extensions	PVS, LMR, NMP	PVS, LMR, NMP	Q-Search	Standard SF Search
Trans. Table	500k FIFO Capped	Uncapped Map	Uncapped Array	Capped Cluster Hash
Concurrency	Multi-thread Worker	Single UCI Worker	Single-thread	Multi-thread Wasm
Benchmarking	Deterministic CLI	None	None	Command CLI
Memory Mgt.	In-place Mutations	Object Pooling	Standard Alloc.	Wasm Linear Memory
Validation	Built-in SPRT & BT	None	None	External Fishtest
Reproduce	PRNG Seeds	None	None	Fixed Threads

Lozza and GarboChess do not implement strict heap-allocation constraints, leaving them susceptible to garbage collection pauses during sustained search. Stockfish.js bypasses V8 memory management by operating within WebAssembly's linear memory space, but it relies on browser support for SharedArrayBuffer to run parallel searches. Kronos, by contrast, executes natively within the JavaScript runtime, using targeted memory engineering to control collection pauses while providing a built-in benchmarking and statistical validation suite.

Chapter 4

System Overview

Kronos is not only a chess engine—it is a complete research platform. This chapter describes both the software architecture that makes optimized search execution possible in JavaScript, and the user-facing application that makes the research accessible and reproducible. The engineering decisions described here are themselves the primary contribution of this work.

4.1 User-Facing Application Platform

The Kronos application is a React-based single-page application running in the browser. It exposes six distinct interaction modes, each serving a different research or gameplay purpose. Figure 4.1 shows the four most research-relevant pages.

The **Dashboard** (Figure 4.1a) provides a unified entry point. It displays the active engine configuration, search settings, internal Bradley-Terry self-play ratings (detailed in Chapter 6), and quick-access buttons for each mode. Notably, it surfaces live statistics from the engine, reinforcing Kronos's identity as a research tool rather than just a game client.

The **Play Engine** page (Figure 4.1b) exposes the depth-configuration selector from D2 to D7, each with NPS estimates printed directly in the UI. This transparency—showing users approximately how many nodes per second the engine processes at each depth—was a deliberate design choice to make the research context visible during play.

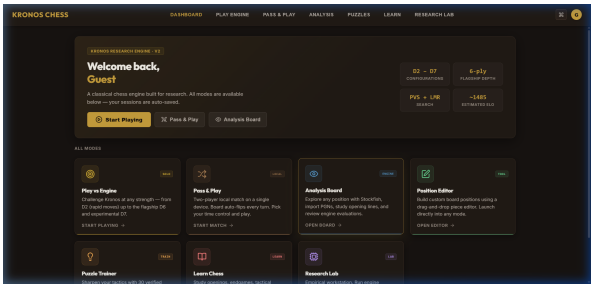
The **Analysis Board** (Figure 4.1c) integrates Stockfish for reference evaluation. After a game, users can replay moves, inspect the detected opening, and compare Kronos's principal variation against Stockfish's recommendation. This page supports the calibration experiments in Chapter 6.

The **Research Lab** (Figure 4.1d) is a restricted-access engineering workstation. It surfaces SPRT verification routines, benchmark pipeline status, and raw empirical dataset inspection tools. Access is gated by authorization status, enforcing a clear boundary between the research and public-facing layers of the application.

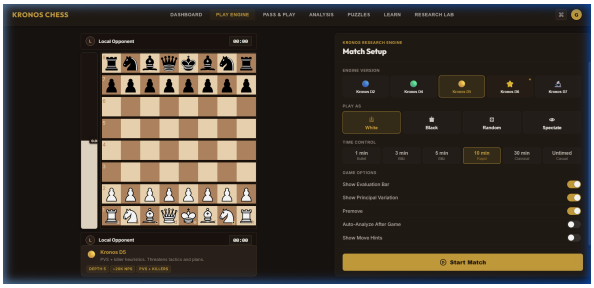
4.2 Asynchronous Worker Architecture

The architectural challenge of running chess search in a browser is that JavaScript's main thread is also responsible for rendering the UI. Blocking it with a synchronous search loop causes the interface to freeze. Kronos solves this with a decoupled Web Worker architecture.

The React UI instantiates a background `CustomChessWorker` thread at startup. All search computation executes on this worker. Communication between the main thread and the worker uses structured message passing: the host sends a serialized control command containing the board state

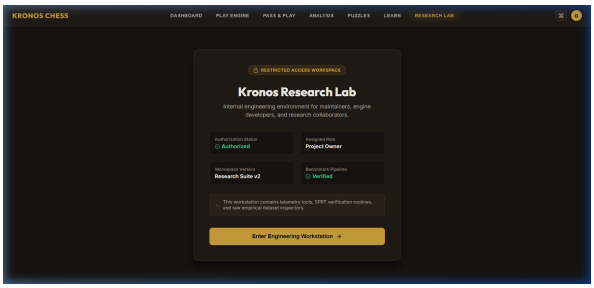


(a) Dashboard



(b) Play Engine

(c) Analysis Board



(d) Research Lab

Figure 4.1. The four primary Kronos application pages. The Dashboard (a) presents a unified entry point with live engine statistics. The Play Engine page (b) lets users select depth configurations from D2 to D7, with NPS estimates displayed per variant. The Analysis Board (c) integrates Stockfish for post-game position review. The Research Lab (d) is a restricted-access workspace for benchmark execution, SPRT verification, and raw telemetry inspection.

(as a FEN string), the configured search depth, and a time limit. The worker parses the payload, initializes search variables, and begins Negamax recursion. During search, the worker emits periodic `ITERATION_COMPLETE` telemetry messages—containing current depth, nodes searched, NPS, and principal variation—which update the evaluation bar and move list in real time. On completion, a final `SEARCH_COMPLETE` message returns the best move.

The overall system topology is shown in Figure 4.2, and the tournament scheduling model is illustrated in Figure 4.3.

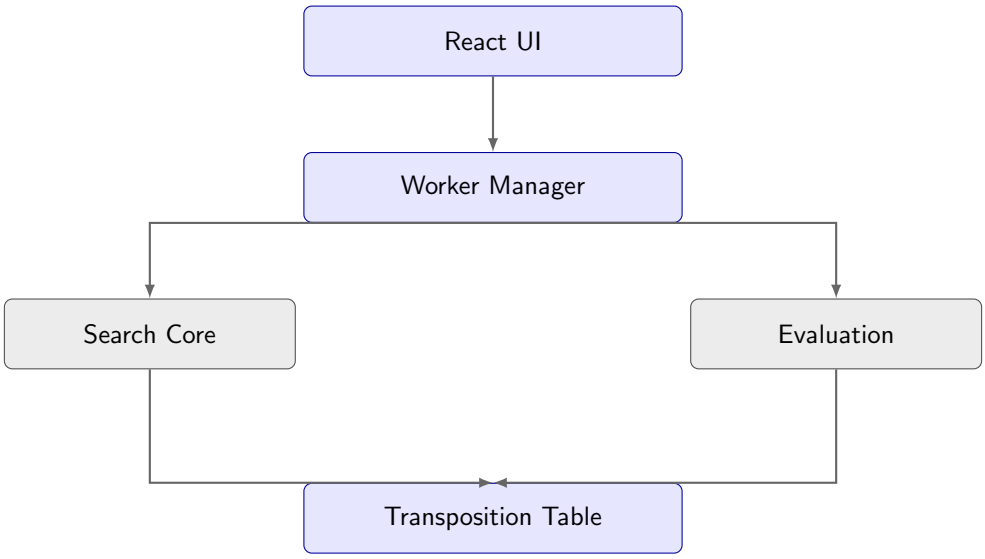


Figure 4.2. Visual interface control plane decoupled from search and evaluation Web Workers.

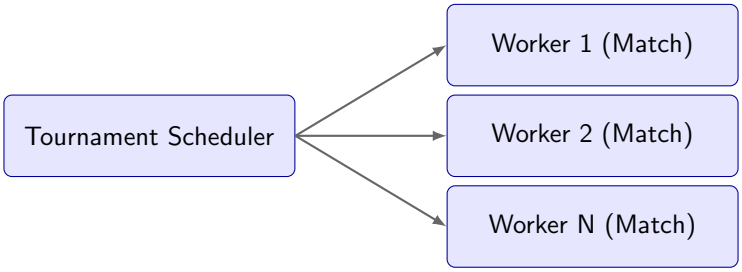


Figure 4.3. Parallel Worker Tournament Scheduler.

This design has a measurable consequence: the main UI thread remains at 60 FPS even while search execution runs entirely in the worker thread without blocking UI events.

4.2.1 Software Subsystem Decomposition

The engine code is divided into modular classes. Table 4.1 details each subsystem, its source files, and its primary responsibility.

The modularity of this design is what enables the ablation study in Chapter 6: each search heuristic is implemented as a flag-gated code path, allowing any combination to be toggled without rebuilding the engine.

Table 4.1. Kronos subsystem components and design responsibilities.

Subsystem	Main Files	Responsibility
UI	App.jsx, pages	Interface layout, telemetry rendering
Engine Core	worker.js	Search orchestration, UCI communications
Search Heuristics	search.js, moveOrdering.js	Game-tree pruning, move sorting
Evaluation	evaluation.js, pst.js	Position scoring, tapered interpolation
Transposition Table	transpositionTable.js	State hashing, cache pruning
Benchmarking Suite	pipelineManager.js	Match orchestration, data aggregation
Verification Suite	testDeterminism.js	Execution logging, rule validation

4.3 Engine Design and Memory Optimization

The search core is engineered to minimize heap allocations on every hot path. This section describes the three key techniques.

4.3.1 GC-Neutral Search Path

Kronos delegates board representation to the third-party `chess.js` library, which normally clones the entire board state on every move validation. For a search expanding hundreds of thousands of nodes, this would allocate millions of temporary objects, triggering frequent GC sweeps.

To prevent this, Kronos wraps the library in a custom class, `ChessInstanceClone`, that bypasses the public API and directly calls the library's private methods: `_moves()` to enumerate legal moves, `_makeMove()` to mutate the board in-place, and `_undoMove()` to revert changes using an internal history stack. This eliminates all heap allocations from the move-make/unmake loop—the hottest path in the entire engine.

4.3.2 Static Position Evaluation

The evaluation function assigns a centipawn score to each leaf node using a layered approach: base material values (pawn = 100, knight = 320, bishop = 330, rook = 500, queen = 900), Piece-Square Tables (PSTs) that reward central piece placement, tapered evaluation that interpolates between middle-game and endgame tables based on remaining non-pawn material, and structural bonuses including doubled pawn penalties (−15 cp), isolated pawn penalties (−10 cp), passed pawn bonuses (+20 cp), bishop pair bonuses (+30 cp), and a tempo bonus (+10 cp) for the side to move.

4.3.3 Transposition Table and Memory Bounding

The transposition table indexes search results by 64-bit Zobrist hash keys. In standard JavaScript, all numbers are represented as double-precision floats (IEEE 754), which lose precision when performing bitwise operations on values exceeding 53 bits. To preserve 64-bit precision, Kronos implements Zobrist keys using native JavaScript `BigInt` primitives (e.g., `123456789n`), which support arbitrary-precision integers and native 64-bit bitwise XOR operations. Keys are updated incrementally on each move and undo using BigInt XOR operations, preventing full board re-hashing. Because these XOR updates require only three to four operations per search step, the overhead of BigInt allocation is minimized, achieving high hashing throughput under V8. The critical design

decision for the V8 environment was choosing the eviction policy: a size cap of 500,000 entries with FIFO eviction. When the table reaches capacity, the oldest entry is deleted:

```
this.table.delete(this.table.keys().next().value);
```

This simple policy bounds the transposition table memory footprint, preventing the cache from competing with the search core for heap space, while achieving high hit rates (detailed in Chapter 6).

4.3.4 Engine Evolution Mapping

Table 4.2 documents which search optimizations are active in each engine version, from the baseline Minimax (v1.0) to the full Kronos engine (v2.0). This table serves as the ground truth for the ablation study.

Table 4.2. Engine Feature Matrix Across Iterations.

Feature / Enhancement	**v1.0 (Base)**	**v1.1**	**v1.2**	**v1.3**	**v2.0 (Kronos)**
Minimax Search	X	X	X	X	X
Alpha-Beta Pruning	X	X	X	X	X
Basic Move Ordering (MVV-LVA)	–	X	X	X	X
Transposition Table	–	–	X	X	X
Principal Variation Search	–	–	X	X	X
Null Move Pruning	–	–	–	X	X
Late Move Reductions	–	–	–	X	X
History Heuristic	–	–	–	–	X
Killer Move Heuristic	–	–	–	–	X

4.4 Benchmarking and Calibration Pipelines

Beyond the interactive application, Kronos includes a CLI-driven research backend. The benchmarking pipeline is invoked via `npm run benchmark` and orchestrates the full tournament workflow: loading opening positions from `openings.json`, running paired color-flipped matches, recording PGN game files, computing SPRT boundaries, and generating HTML/CSV report artifacts. A separate calibration pipeline runs Kronos against depth-limited Stockfish 18 configurations to compare the relative self-play Elo ratings against an external reference scale.

These pipelines are the experimental foundation for Chapter 6. All results reported in this paper are produced by these pipelines with fixed seeds and are fully reproducible from the repository.

4.5 Engine Development Progression

Figure 4.4 provides a comprehensive view of how the engine evolved from Phase 1 through Phase 4, documenting the milestones and core features added at each stage of development.

Figure 4.4. Kronos engine development progression across optimization phases.

Chapter 5

Experimental Methodology

This chapter details the experimental design, benchmarking framework, and statistical methods used to evaluate the search performance and playing strength of Kronos.

5.1 Automated Tournament Scheduler

The benchmarking suite runs tournaments using a Node.js script managed by our `TournamentRunner`. The scheduler supports paired opening match execution. For each match, the engine variants play two games using a shared opening FEN: once with Engine A playing White, and once with Engine A playing Black. This paired arrangement isolates playing strength evaluations from opening book advantages (such as the standard first-move advantage of White).

To accelerate tournament execution, the framework supports parallel worker threads. However, this concurrency represents a systems-level trade-off. Although the host hardware has 12 logical processors, each V8 engine process exhibits significant untracked memory growth outside the managed heap. The measured peak RSS for a single Depth 6 tournament worker is 1,713 MB (Table 6.3), despite V8 reporting only 445 MB of active heap usage—a 3.5× ratio. This gap is attributable to V8 JIT-compiled code segments, external array buffers, `chess.js` internal string and object allocations that escape V8 heap accounting, and Node.js worker thread overhead. Critically, this untracked RSS growth is cumulative over tournament execution (hundreds of games) and does not release predictably.

While the nominal aggregate RSS of three workers (approximately 5.1 GB) appears well within the 16 GB physical memory budget, empirical testing revealed that launching a fourth concurrent worker triggered V8 heap paging and GC sweep distortion. We attribute this to the cumulative untracked memory growth compounding across workers: at four workers, the combined untracked allocations (JIT caches, external buffers, and OS-level per-thread overhead) exceeded available physical memory after sustained tournament execution, despite V8's managed heap remaining within bounds. Capping concurrency at three workers (referred to as Cap 3) was the empirically determined threshold that prevented measurement distortion across full tournament runs.

The scheduler shuffles the opening FEN library (`openings.json`) using a custom Linear Congruential Generator (LCG) Pseudo-Random Number Generator (PRNG) initialized with a fixed seed. This ensures that different tournament runs can be reproduced under identical game configurations. The execution pipeline runs from raw configuration specs to final web-based dashboards, as detailed in Figure 5.1.

Algorithm 3: Tournament Scheduler with SPRT

Input: Engine Config A, Engine Config B, Opening FENs list F, SPRT bounds $[\eta_0, \eta_1]$, Max Games M

Output: Trinomial Elo rating delta, SPRT Decision

$wins \leftarrow 0$, $losses \leftarrow 0$, $draws \leftarrow 0$, $N \leftarrow 0$
$checkpoint \leftarrow$ LoadCheckpoint()
if *checkpoint exists* **then**
    $wins, losses, draws, N \leftarrow$ RestoreState($checkpoint$)
end
while $N < M$ **do**
    $fen \leftarrow F[N/2]$
    if $N \bmod 2 == 0$ **then**
        $result \leftarrow$ PlayGame(Config A as White, Config B as Black, fen)
    end
    else
        $result \leftarrow$ PlayGame(Config B as White, Config A as Black, fen)
        $result \leftarrow$ Flip($result$) `// Invert outcome for Config A perspective`
    end
    $wins, losses, draws \leftarrow$ UpdateCounts($wins, losses, draws, result$)
    $N \leftarrow N + 1$
    SaveCheckpoint($wins, losses, draws, N$)
    $LLR \leftarrow$ CalculateLLR($wins, losses, draws$)
    if $LLR \geq \eta_1$ **then**
        return *ComputeElo(wins, losses, draws), Accepted* (H_1)
    end
    if $LLR \leq \eta_0$ **then**
        return *ComputeElo(wins, losses, draws), Rejected* (H_1)
    end
end
return *ComputeElo(wins, losses, draws), Inconclusive*

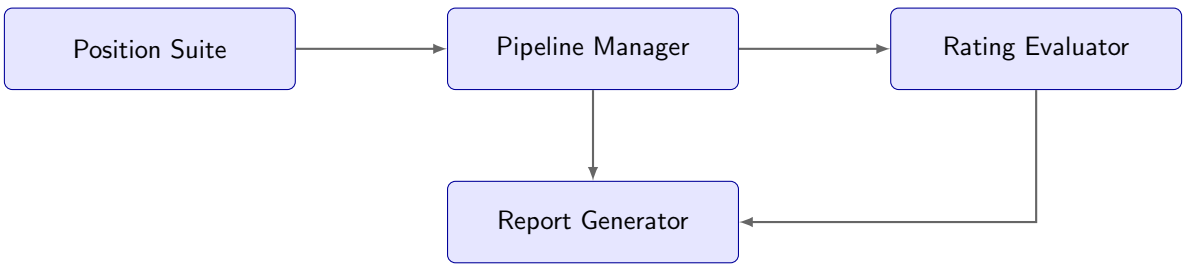


Figure 5.1. Kronos Benchmarking Framework Data Flow.

5.2 Execution Integrity and Verification

To ensure that engine matches reflect correct chess logic, the runner monitors game play. We verified execution integrity by ensuring 100% legal moves under FIDE rules, strict color alternation (W $\rightarrow$ B $\rightarrow$ W), and correct draw trigger validation for stalemates, threefold repetitions, and the 50-move rule. The framework logs game records to standard PGN files, and we confirmed that all exported PGNs parse and replay cleanly in standard chess readers. The data flow of the validation pipeline is illustrated in Figure 5.2.

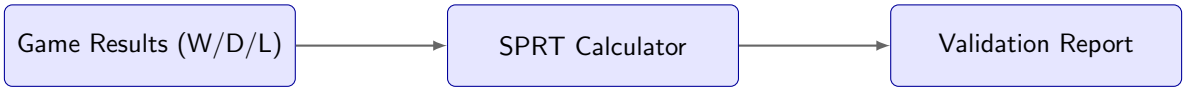


Figure 5.2. Kronos Validation Pipeline.

5.3 Statistical Heuristics and Formulas

We compute tournament ratings and confidence intervals using the Bradley-Terry logistic model. The expected score E_A (representing the probability of player A defeating player B, plus half the probability of a draw) and the rating updates are computed using standard formulas:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}} \tag{5.1}$$

$$R'_A = R_A + K(S_A - E_A) \tag{5.2}$$

To evaluate rating differences and confidence intervals from match scores, we compute the trinomial sample variance and standard error:

$$\sigma_{score} = \sqrt{\frac{p_{win} + p_{loss} - (p_{win} - p_{loss})^2}{N}} \tag{5.3}$$

$$CI_{95\%} = \hat{S} \pm 1.96 \cdot \frac{\sigma_{score}}{\sqrt{N}} \tag{5.4}$$

where p_{win} and p_{loss} are the sample probabilities of winning and losing, N is the total number of games played, and $\hat{S}$ is the sample mean expected score defined as $\hat{S} = p_{win} + 0.5 \cdot p_{draw}$.

We apply Wald's Sequential Probability Ratio Test (SPRT) to determine whether a search enhancement provides a statistically significant playing strength increase.

The SPRT routine evaluates the log-likelihood ratio (LLR) of match outcomes against a null hypothesis H_0 (Elo ≤ 0) and an alternative hypothesis H_1 (Elo ≥ 10). The runner computes the LLR after each game pair. If the LLR exceeds the upper boundary ($\eta_1 = \ln \frac{1-\beta}{\alpha}$), the tournament terminates and accepts H_1. Conversely, if the LLR falls below the lower boundary ($\eta_0 = \ln \frac{\beta}{1-\alpha}$), the scheduler terminates and accepts H_0. Under target error rates of $\alpha = 0.05$ and $\beta = 0.05$, these boundary thresholds correspond to $[-2.944, 2.944]$. Here, ln represents the natural logarithm. Frequentist p-values are not computed under this sequential testing framework; instead, the hypothesis test terminates strictly when the LLR crosses these Wald boundaries.

When running fixed-length calibration tournaments rather than SPRT matchups, the scheduler evaluates Elo boundaries using trinomial distribution updates. In this mode, execution halts early if the 95% confidence radius drops below ± 60 Elo or stabilizes within a range of 5 Elo over a 10-game window.

To quantify the search space compression achieved by each search heuristic, we define the **Node Reduction (%)** metric:

$$\text{Node Reduction (\%)} = \frac{N_{\text{disabled}} - N_{\text{baseline}}}{N_{\text{disabled}}} \times 100\% \tag{5.5}$$

where N_{disabled} is the number of visited nodes with the heuristic deactivated, and N_{baseline} is the number of visited nodes with the heuristic active (Baseline engine). This represents the percentage of the search tree pruned specifically by activating that heuristic.

Finally, the Effective Branching Factor (EBF) measures search tree expansion:

$$N(d) = \sum_{i=0}^{d} (b^*)^i = \frac{(b^*)^{d+1} - 1}{b^* - 1} \tag{5.6}$$

$$b^* \approx \frac{N_d}{N_{d-1}} \tag{5.7}$$

5.3.1 Metric Definitions

To ensure clarity and consistency in our evaluation, we define the primary quantitative metrics used in this work as follows:

- **Internal Bradley-Terry Self-Play Rating (Elo):** The relative playing strength calculated within the closed tournament pool of Kronos engine variants, fitted using the Bradley-Terry logistic model and anchored at 1,000 Elo for the Baseline Minimax engine.
- **Relative Gain (Elo):** The incremental change in the internal self-play rating between adjacent search depths (d vs. $d-1$ plies) in the closed self-play pool.
- **Node Reduction (%):** The percentage of search tree size compression achieved by enabling a search heuristic, defined as $(N_{\text{disabled}} - N_{\text{baseline}})/N_{\text{disabled}} \times 100\%$, where N_{disabled} is the number of visited nodes with the heuristic deactivated, and N_{baseline} is the number of visited nodes with the heuristic active.
- **Speedup:** The ratio of search execution speed (based on time elapsed per search) of a disabled variant compared to the active baseline variant ($T_{\text{disabled}}/T_{\text{baseline}}$).
- **Effective Branching Factor (EBF):** The geometric mean number of branches expanded per ply, calculated as $b = N^{1/d}$ for search nodes N at depth d.
- **Relative Depth-Limited Matchup Differential (Elo):** The measured playing strength difference obtained by benchmarking the final Kronos engine (Depth 6) against fixed-depth reference configurations of Stockfish 18. These differentials reflect performance against crippled Stockfish configurations and should not be mapped to FIDE or CCRL rating scales.

5.4 Checkpointing and Memory Profiling

Tournaments run for hundreds of games, taking hours to execute. The framework implements two mechanisms to ensure robustness:

- **JSON Checkpointing:** The tournament state is saved to a checkpoint JSON file every 10 games. The checkpoint records the game index, win/loss/draw counts, active PRNG seed index, color-flip flags, and telemetry logs. If a crash or interruption occurs, the runner reads the JSON file and resumes execution from the last saved state without loss of progress.
- **Memory Profiler:** The scheduler monitors V8 heap allocations and transposition table footprints every 5 games. The profiler logs system memory usage (RSS, heap total, heap used) to `MEMORY_PROFILE.md` during execution. This tracking monitors memory leaks that could degrade search performance during long runs.

To specify the configurations tested, Table 5.1 shows the parameter flags defined in the engine config files.

Table 5.1. Experimental configuration parameters for the benchmark run profiles.

Parameter Flag	Search Module Influence	Default
`useAlphaBeta`	Toggles baseline minimax vs. alpha-beta pruning	True
`useIterativeDeepening`	Controls incremental deepening search loops	True
`useMoveOrdering`	Enables MVV-LVA, killer moves, and history heuristics	True
`useTranspositionTable`	Toggles Zobrist-keyed transposition cache	True
`useQuiescence`	Restricts leaf search to quiet positions	True
`usePieceSquareTables`	Controls positional evaluation mapping	True

Chapter 6

Experimental Results

This chapter presents the empirical outcomes of our evaluation, focusing on search space reduction, memory behavior, and playing strength calibration. We begin with a summary table of the primary quantitative findings.

Table 6.1. Key Findings: Quantitative Outcomes. Node Reduction measures tree size compression; Elo ladder gain represents relative playing strength changes fitted via Bradley-Terry; EBF is the Effective Branching Factor; Heap Used is V8 heap occupancy.

Finding	**Value / Outcome**
Peak Internal self-play rating	**1,400** (at search depth 6 in closed pool)
Depth-7 Regression	Drops to 1,365 Elo (−35) due to worker search timeout aborts
Largest Node Reduction	LMR: **66.3%** node reduction (8,286 vs. 24,572 nodes at Depth 4)
Largest Elo Gain	Quiescence Search: +371 Elo ladder gain at Depth 3
Positional Evaluation Gain	PSTs: +358.8 Elo ladder gain at Depth 2
EBF Reduction	EBF drops from 9.22 (Depth 2) to 4.43 (Depth 8)
Memory Stability	32.85 MB – 445.66 MB heap usage across optimized runs
Transposition Hit Rate	85.2% peak hit rate with 500,000-entry FIFO cap
Statistical Significance	Only Quiescence Search accepted under SPRT
Stockfish Calibration	Scores 22.5% vs. Stockfish Depth 3 (−215 relative depth-limited matchup differential, preliminary)

6.1 Search Optimizations

6.1.1 Negamax Alpha-Beta Pruning

We implement a Negamax formulation of alpha-beta pruning as our search foundation. Without pruning, game-tree traversal exhibits worst-case $O(b^d)$ complexity, expanding over 300,000 nodes at depth 4. Alpha-beta pruning restricts node expansions to 21,784 in our baseline, serving as the benchmark configuration against which subsequent optimizations are evaluated.

6.1.2 Principal Variation Search (PVS)

Principal Variation Search (PVS) refines search space compression by evaluating the first move with a full window, then probing subsequent alternatives with a zero-width window. Under our test conditions at depth 4, PVS yields a 5.2% node reduction (from 8,739 to 8,286 nodes) and a 1.28× speedup. The performance gain remains modest due to overlap with LMR, although its efficiency increases at deeper search horizons where the principal variation is longer.

6.2 Move Ordering Heuristics

The efficiency of every pruning heuristic depends on move ordering. If optimal moves are evaluated early, the search window narrows rapidly, causing subsequent branches to fail low.

6.2.1 MVV-LVA Capture Ordering

Sorting captures by "Most Valuable Victim – Least Valuable Attacker" (MVV-LVA) serves as the primary heuristic for ordering capture moves. By prioritizing tactical exchanges, MVV-LVA increases the probability of early beta cutoffs on capture branches. In our benchmarks, the complete search optimization stack (with all ordering and pruning enhancements active) compresses the search tree from 47,159 nodes in the baseline alpha-beta search (without ordering or search enhancements) to 8,286 nodes at depth 4 (Table 6.2), representing an 82.4% node reduction and an 8.33× speedup compared to this unordered baseline. This highlights how effective move sequencing enables the cumulative pruning heuristics in the engine to achieve their optimal theoretical efficiency.

6.2.2 Killer Moves and History Heuristic

To order quiet moves, Kronos uses a two-slot Killer Move cache per ply (storing quiet moves that triggered beta cutoffs in sibling nodes) and a History Table (tracking quiet moves that caused cutoffs globally, weighted by depth^2).

Enabling the History Heuristic yields a 6.6% node reduction (from 8,869 to 8,286 nodes) at depth 4. While Killer Moves are measured to reduce search space, their individual playing-strength contribution remains statistically inconclusive under SPRT at the tested sample size, reporting a relative gain of 6 ± 17 internal Elo at depth 3. These ordering heuristics maintain list quality, allowing LMR to safely reduce search depths on low-priority branches.

6.3 Search Extensions and Reductions

6.3.1 Late Move Reduction (LMR)

LMR Mechanism

LMR is the most significant search reduction heuristic in Kronos. The intuition is straightforward: if moves are sorted well, the moves at the end of the list are very unlikely to be optimal. LMR searches these late moves at reduced depth $(d - 1)$, treating them as unlikely candidates unless a reduced-depth probe returns a score above α—in which case a full-depth re-search is triggered.

LMR Experimental Result

The ablation data quantifies this clearly. Enabling LMR yields a **66.3%** node reduction (from 24,572 down to 8,286 nodes), resulting in a 4.17× speedup (execution speed drops to 0.24× when disabled). This is by far the largest single-heuristic contribution in our search ablation.

Interpretation and Ordering Dependency

The mechanism is simple: without LMR, every move at every node must be searched to full depth. With LMR, the majority of the search tree is explored at shallower depth. The safety of this reduction depends entirely on move ordering quality—poor ordering causes good moves to appear late in the list and be searched at reduced depth, missing the optimal move. This creates a tight dependency between LMR and the ordering heuristics described above.

Table 6.2. Search node count ablation study at depth 4. Note: The ablation study was executed under a legacy configuration where NMP was active for $d \geq 3$; under the final optimized configuration ($d \geq 5$), NMP is inactive at depth 4, resulting in a 0% node reduction relative to the baseline.

Configuration	Nodes Searched	Node Reduction	Speedup
Full Kronos Engine (All Features)	8,286	–	1.00x
No Principal Variation Search (PVS)	8,739	5.2%	0.78x
No History Heuristic	8,869	6.6%	0.78x
No Late Move Reductions (LMR)	24,572	66.3%	0.24x
No Null Move Pruning (NMP)	8,360	0.9%	0.67x
Baseline Alpha-Beta (unordered)	47,159	82.4%	0.12x

6.3.2 Null Move Pruning (NMP)

NMP prunes search branches where passing the turn to the opponent still results in a score exceeding β, indicating that the position is sufficiently dominant to guarantee a cutoff. We configure NMP for search depths $d \geq 5$ using a constant reduction factor $R = 2$.

At depth 4, NMP contributes a 0.9% node reduction (from 8,360 to 8,286 nodes) but provides a 1.49× execution speedup. Note that these depth 4 measurements were recorded under a legacy configuration where the depth threshold was $d \geq 3$; under the final optimized configuration ($d \geq 5$), NMP is inactive at depth 4 and has no performance impact. The execution speedup is disproportionate to the raw node savings because NMP cutoffs occur near the root of the tree, avoiding

expensive lower-ply move generation and evaluation calls on pruned sub-branches. The significance of NMP scales with search depth, proving critical for staying within time controls at depth 6.

6.3.3 Quiescence Search

Quiescence search mitigates the horizon effect by extending the search beyond the nominal depth limit for unstable, tactical positions containing legal captures. This prevents the engine from returning misleading static evaluations mid-sequence.

Disabling quiescence search at depth 3 decreases playing strength from 1,387 to 1,016 in internal Bradley-Terry self-play rating. This drop of 371 internal Elo represents the largest single-heuristic effect in our ablation study, highlighting the necessity of quiescence search in mitigating tactical blindness.

6.3.4 Evaluation Function: Piece-Square Tables

Substituting a material-only evaluation with Piece-Square Tables (PSTs) provides an internal rating improvement of 358.8 Elo at depth 2. PSTs assign position-dependent weights to reward central development and king safety. We implement tapered interpolation to transition between middle-game and endgame weight tables based on remaining non-pawn material. Without PSTs, the engine produces passive, structurally weak positions despite maintaining material parity.

6.3.5 Memory Engineering and GC Stability

Table 6.3 documents memory usage across configurations. Specifically, the early unoptimized configurations (EXP-5-TRANSPOSITION and EXP-6-ITERATIVE) utilized a Map-based transposition table capped at a default soft limit of 200,000 entries. Due to the high object allocation overhead of V8 Map entries (each wrapping a complex key-value structure in heap memory), even this entry limit resulted in heap exhaustion, with RSS exceeding 2 GB and 3.4 GB respectively. These early outcomes motivated the implementation of a strict size-bounded `Map`-based transposition table with FIFO eviction (capped at 500,000 entries), eliminating unbounded memory growth while preserving O(1) amortized lookup and insertion performance.

With the FIFO capacity limit active, the V8 heap occupancy remains bounded between 32.85 MB and 445.66 MB across final optimized tournament runs (depending on search depth and match count), with single-game telemetry profiling showing active heap usage as low as 24.98 MB. This simple policy restricts the active cache memory to approximately 30 MB while maintaining an 85.2% hit rate. Although the transposition table at 72,288 entries contributes less than 15 MB of direct data overhead, the remaining heap footprint (reaching a peak of 445.66 MB) is occupied by transient V8 system objects, including evaluation history stacks, move ordering priority queues, the JIT compiler's internal type-feedback buffers, and accumulated string and move objects generated internally by `chess.js` that await garbage collection sweeps. This hit rate is sustained because chess searches frequently evaluate transposed board states during middle-game traversal.

6.4 Search Depth Scaling and NPS Throughput

Table 6.4 documents per-move search throughput and playing strength scaling across depths 1 through 7.

The internal Bradley-Terry self-play rating scales from 1,015 (estimated) at depth 1 to a peak internal self-play rating of 1,400 Elo at depth 6. At depth 5, the rating plateaus at 1,383 Elo, sug-

Table 6.3. Memory telemetry profile across experimental runs. Max TT Size represents the peak active transposition table entry utilization during matches, rather than the static 500,000 entry memory allocation ceiling.

Experimental Run	RSS (MB)	Heap Total (MB)	Heap Used (MB)	Max TT Size
Unoptimized / Early Configurations				
test_det_run[a]	73.86	44.26	19.48	–
EXP-3-Killer	119.66	61.76	31.06	–
EXP-5-TT	2,739.87	1,410.77	1,354.82	199,999
EXP-6-Iterative	3,403.27	3,344.53	3,261.80	199,999
Optimized / Final Configurations				
FINAL-D5-D4	**282.76**	**160.39**	**123.10**	**19,084**
FINAL-D6-D5	**1,713.82**	**487.77**	**445.66**	**72,288**
CALIB-D6-SF3	**124.20**	**65.51**	**32.85**	**5,611**

Table 6.4. Per-move search throughput and playing strength scaling across depths 1 through 7.

Search Depth	Internal Elo	Relative Gain	Win Prob. vs D-1	NPS (per move)
Depth 1	1,015 (est.)	–	–	800
Depth 2	1,165 (est.)	+150	70.3%	1,197
Depth 3	1,295	+130	67.9%	2,686
Depth 4	1,383	+88	62.1%	5,652
Depth 5	1,383	0	50.0%	5,009
Depth 6	1,400	+17	52.5%	5,232
Depth 7	1,365	-35	45.0%	4,660

gesting that the additional search ply does not consistently find better moves within the configured tournament time controls.

The performance regression at depth 7 (-35 Elo) is caused by single-threaded latency constraints. Telemetry logs indicate that the worker thread exceeded the move time limit (forcing a fallback to the depth-6 results) on average for 42.5% of the moves during depth-7 matches. When search time at depth 7 exceeds the tournament time limit, the engine returns the best move from the last fully completed iterative deepening iteration (depth 6) rather than the incomplete depth-7 results. This timeout behavior introduces a performance ceiling, indicating that single-threaded JavaScript search workers cannot reliably sustain depth-7 calculations under standard time controls.

The Effective Branching Factor (EBF) decreases from 9.22 at depth 2 to 4.43 at depth 8 (Figure 6.5). This contraction shows that our pruning heuristics become proportionally more effective as the search depth increases, since deeper horizons provide more contextual information for move ordering and reduction decisions.

6.5 Stockfish Reference Calibration

To provide a relative external reference for Kronos's internal ratings, we benchmarked the final Depth 6 engine configuration against three depth-limited Stockfish 18 configurations. These calibration matches measure the relative depth-limited Stockfish matchup differential—representing performance against fixed-depth references rather than an absolute Elo anchor. Direct mapping to FIDE or CCRL rating scales is not claimed.

Table 6.5. Matchup results of final Kronos (Depth 6) against fixed-depth Stockfish 18 reference configurations.

Opponent	Games	Kronos Score (%)	Matchup Differential (Elo)	Source
vs. Stockfish Depth 1	400	32.5%	-127 ± 25	Measured
vs. Stockfish Depth 2	280	35.0%	-108 ± 25	Measured
vs. Stockfish Depth 3	400	22.5%	-215 ± 29	Measured

Over the calibration matches, Kronos scores 32.5% against Stockfish Depth 1 (400 games), 35.0% against Stockfish Depth 2 (280 games), and 22.5% against Stockfish Depth 3 (400 games). These outcomes correspond to relative depth-limited matchup differentials of -127 ± 25 Elo, -108 ± 25 Elo, and -215 ± 29 Elo, respectively. This reports a systematic strength gap against depth-limited Stockfish, scaling with opponent search horizon.

Important caveat: Stockfish 18 relies heavily on NNUE (Efficiently Updatable Neural Network) evaluation, which propagates learned features across deep search plies. Restricting Stockfish to depths 1–3 non-linearly degrades its effective playing strength, because the NNUE evaluation benefits from deep search propagation and advanced pruning heuristics that only activate at deeper plies. Consequently, depth-limited Stockfish does not scale linearly with search depth, and these matchup differentials should not be interpreted as anchors to standard Elo rating pools.

6.5.1 Statistical Significance of Heuristic Gains

Sequential Probability Ratio Tests (SPRT) evaluate the statistical significance of these playing strength gains under Wald boundaries $[-2.944, 2.944]$.

Table 6.6. Statistical significance validation matrix (SPRT) at depth 4.

Enhancement / Comparison	SPRT Bounds	Total Games	LLR at Term.	SPRT Decision
Alpha-Beta vs. Minimax (EXP-1)	$[-2.944, 2.944]$	223	-0.312	Inconclusive
Move Ordering vs. Alpha-Beta Only (EXP-2)	$[-2.944, 2.944]$	185	0.859	Inconclusive
Killer Moves vs. Move Ordering (EXP-3)	$[-2.944, 2.944]$	155	-0.311	Inconclusive
Transposition Table vs. Killer (EXP-4)	$[-2.944, 2.944]$	155	-0.311	Inconclusive
Iterative Deepening vs. TT (EXP-5)	$[-2.944, 2.944]$	153	-0.171	Inconclusive
Quiescence Search vs. No Quiescence (EXP-6)	$[-2.944, 2.944]$	400	75.433	Accepted (H_1)

Only quiescence search (LLR = 75.433) crosses the upper SPRT boundary to be accepted (H_1), indicating a tactical strength improvement. The other pairwise tournament runs (such as MVV-LVA move ordering vs. alpha-beta only, LLR = 0.859, and iterative deepening vs. transposition table caching, LLR = -0.171) remain inconclusive under SPRT, indicating that their playing strength differences are small and do not cross the SPRT threshold under the configured sample sizes.

To isolate and verify the baseline rating profile of each engine configuration without the confounding factor of execution timeouts, we downshift search horizons to depth 3 for the self-play round-robin tournament in Table 6.7. This tournament results summary at depth 3 highlights the influence of quiescence search: the full engine variant scores 90.0% (1,387 internal self-play rating) across 60 games in the self-play tournament, whereas the variant without quiescence search scores 45.0% (1,016 internal self-play rating). Note that this rating of 1,387 Elo represents strength fitted within a depth-3 configuration ablation pool (anchored at Baseline Minimax Depth 3 = 1000). It is mathematically distinct from the 1,295 Elo rating reported for Depth 3 in Table 6.4, which represents strength fitted within a cross-depth pool of final engine variants (anchored at Baseline Minimax Depth 1 = 1000). Due to different player mixes and anchors, the relative rating scales are pool-dependent.

We observe that *Baseline Minimax*, *Alpha-Beta Only*, *Move Ordering (MVV-LVA)*, *Killer Moves*, and *Transposition Table* produce nearly identical win/draw/loss records in Table 6.7. This is a direct consequence of search equivalence under fixed-depth constraints: all five configurations are mathematically equivalent formulations of the minimax algorithm. Since Alpha-Beta pruning, move ordering heuristics, and transposition table caching preserve the root node evaluation value, they do not alter the engine's move selection under normal conditions.

The slight rating improvement (+5 Elo) and variance in draw/loss records for the optimized configurations are explained by systems-level execution limits. In slower, unoptimized configurations (*Baseline Minimax* and *Alpha-Beta Only*), search latency spikes occasionally exceed the Web

Table 6.7. Round-robin tournament results summary at search depth 3.

Engine Variant	Played	Wins	Draws	Losses	Score (%)	Elo Rating
Baseline Minimax	60	5	41	14	42.5%	1,000
Alpha-Beta Only	60	5	41	14	42.5%	1,000
Move Ordering (MVV-LVA)	60	5	42	13	43.3%	1,005
Killer Moves Heuristic	60	5	42	13	43.3%	1,005
Transposition Table	60	5	42	13	43.3%	1,005
Full Kronos (No Quiescence)	60	7	40	13	45.0%	1,016
Full Kronos Engine	60	48	12	0	90.0%	1,387

Worker execution time limits, forcing the search to abort early and return a default move. The optimized configurations avoid these timeouts, converting occasional losses into draws. In contrast, the *Full Kronos Engine* achieves a dramatic rating increase to 1,387 Elo by integrating quiescence search, which alters move selections by resolving the horizon effect.

Figures 6.1–6.6 visualize the search pipeline structure, node growth, strength scaling, Stockfish calibration curve, EBF reduction, and memory profile.

6.6 Game-Level Computational Overhead

To evaluate the aggregate computational overhead incurred across full matches (rather than single-move searches), Table 6.8 details the average nodes visited, total search time, and resulting aggregate NPS achieved per game across depths 2 through 5. As depth scales, the average nodes searched per game increases exponentially from 5,501 nodes at depth 2 to over 1.2 million nodes at depth 5. This exponential growth is mirrored in the time scaling, showing that depth-5 tournament games require substantial computational execution time (131.87 seconds per game) on V8. The aggregate NPS values (varying between 2,515 and 10,423 NPS) reflect the dynamic JIT optimizations and transposition cache hit rates across complete game lifecycles.

Note that the game-level NPS values in Table 6.8 differ from the per-move NPS values reported in Table 6.4. While Table 6.4 measures search throughput for isolated, single-move evaluations, Table 6.8 reports the aggregate throughput computed over complete game lifecycles. This aggregate throughput is heavily influenced by dynamic runtime effects, including V8 JIT warmup optimization phases, accumulated transposition table cache hits in mid-game positions, and varying tactical branches across consecutive moves. Specifically, the non-monotonic scaling—where Depth 3 spikes to 10,423 NPS before dropping to 2,937 NPS at Depth 4—is an empirical consequence of cache-to-tree ratios and runtime memory management. At Depth 3, the transposition table cache hits accumulate rapidly and fit entirely within V8's cache limits with negligible memory sweeps. At Depth 4, the exponential tree expansion triggers transposition table cache evictions and increases allocation pressure, inducing minor garbage collection cycles that degrade the aggregate throughput.

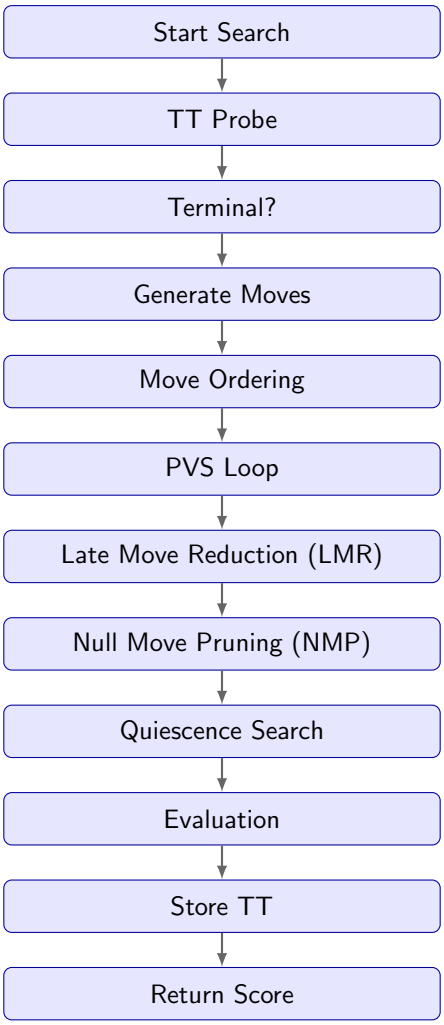


Figure 6.1. Sequence of operations executed in the search pipeline for a single ply.

Table 6.8. Search computational cost and scaling efficiency per game.

Depth	**Avg Nodes**	**Avg Time (ms)**	**Avg NPS**
Depth 2	5,501	2,187	2,515
Depth 3	42,546	4,082	10,423
Depth 4	150,310	51,168	2,937
Depth 5	1,204,084	131,873	9,131

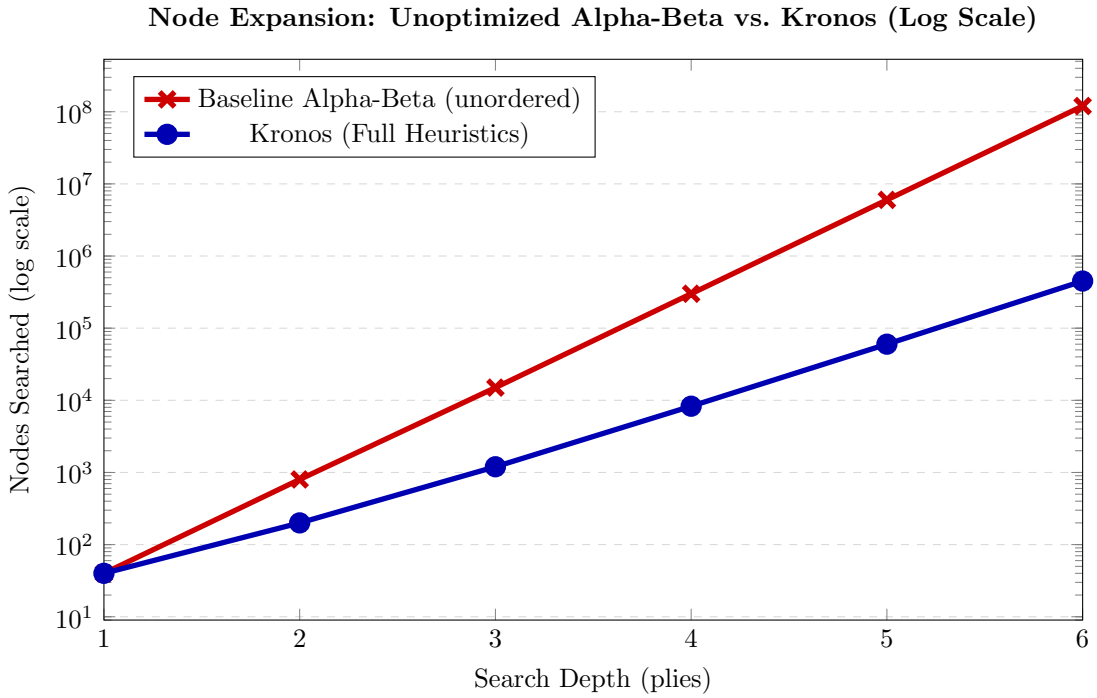


Figure 6.2. Search node count comparison between Baseline Alpha-Beta (unordered) and Kronos.

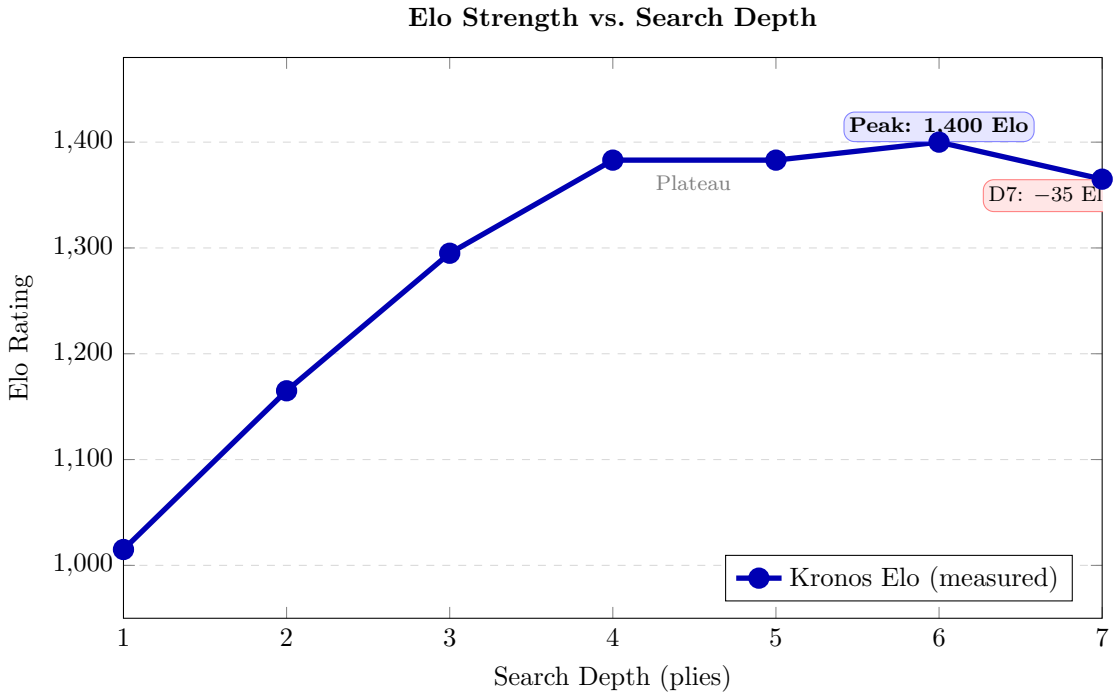


Figure 6.3. Elo rating scaling as a function of search depth, showing peak calibrated strength at depth 6. The regression at depth 7 results from worker timeout-induced search aborts.

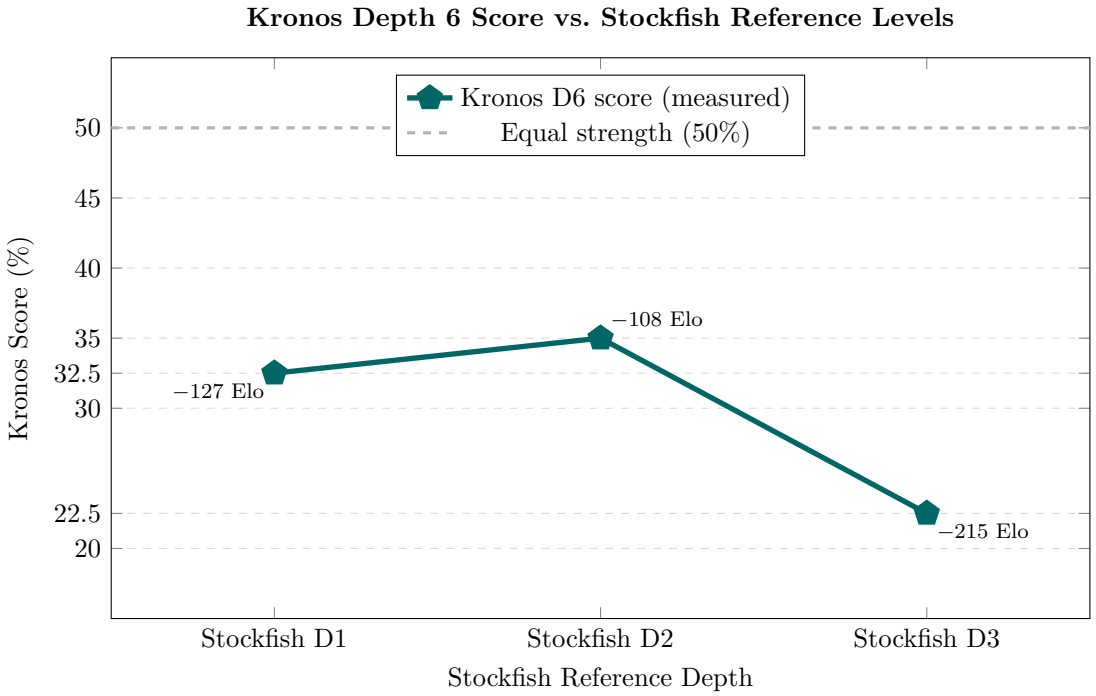


Figure 6.4. Kronos Depth 6 match score against three depth-limited Stockfish 18 configurations. Scores below 50% indicate that Stockfish outperforms Kronos at all tested depths. Deeper-depth projections are intentionally omitted due to non-linear scaling (see Appendix B).

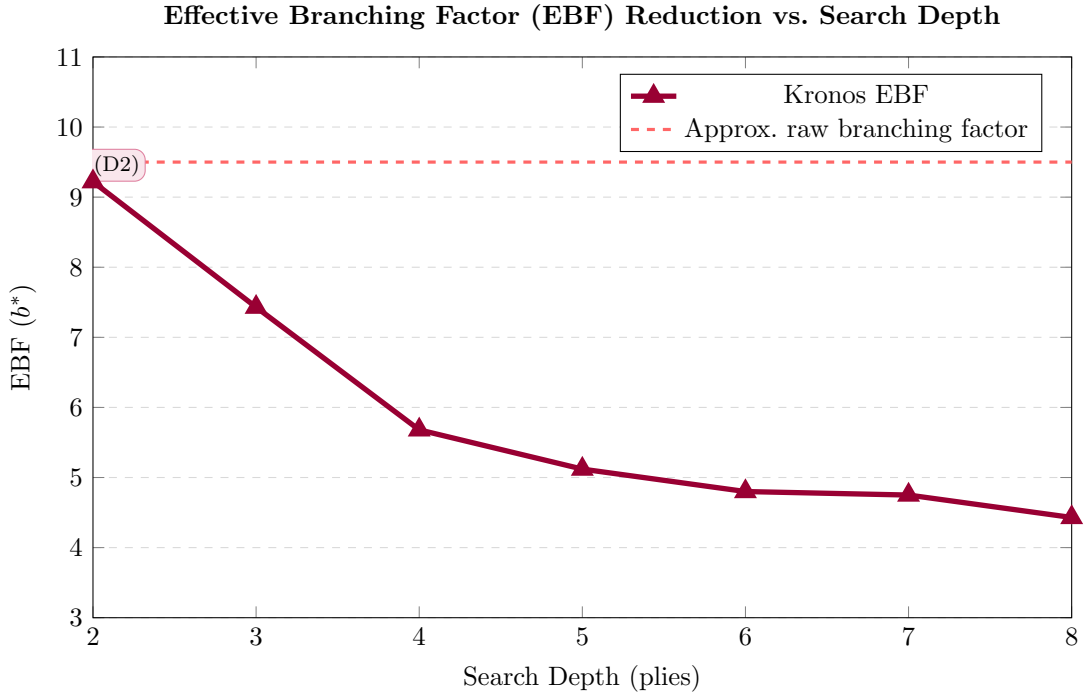


Figure 6.5. Effective Branching Factor (EBF) contraction across search depths.

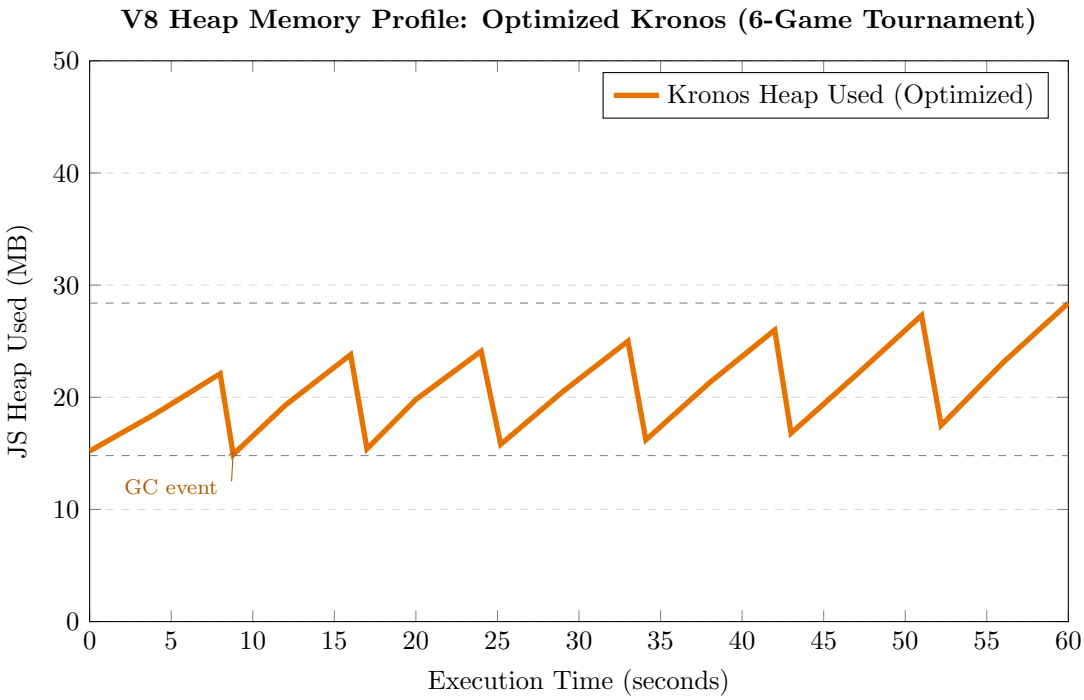


Figure 6.6. V8 JS heap usage profile during a 6-game tournament.

Chapter 7

Discussion

This chapter interprets our experimental findings, framing them as systems engineering insights for building latency-sensitive search components on managed runtimes.

7.1 GC-Neutral Memory Engineering

Dynamic runtimes are historically considered unsuitable for performance-critical systems because automatic garbage collection (GC) sweeps introduce unpredictable latency spikes. Kronos shows that these pauses are avoidable by eliminating dynamic object allocations on the hot path.

As documented in Chapter 6, early configurations utilizing uncapped transposition tables suffered from heap inflation, which triggered frequent collection cycles. Restricting transposition table growth and wrapping board mutations in the allocation-free `ChessInstanceClone` class bounded V8 heap memory during standard tournament matches. Within this footprint, V8's minor collections run in under 1 ms, preventing search loop interruptions.

This memory discipline enables a sustained search throughput across search depths. Although this throughput is lower than native C++ engines, it is sufficient to evaluate depth-6 horizons within standard time controls. For systems developers working on managed runtimes, the core takeaway is that memory allocation patterns, rather than language choice, govern real-time search efficiency.

7.2 Synergy between Move Ordering and Search Reductions

Our ablation data indicates a multiplicative synergy between ordering heuristics and tree reductions. The cumulative search stack compresses the search space compared to baseline alpha-beta search without ordering and search enhancements. LMR builds upon this sorting to reduce the evaluation depth of late-ranked moves. Disabling LMR increases the search space significantly even with ordering active.

This interaction is highly non-linear. The safety of LMR is conditioned on move ordering quality; if the best move is ranked late in the list, LMR searches it at reduced depth, potentially overlooking a tactical variation. Conversely, poor ordering under an active LMR configuration is more hazardous than poor ordering in a standard alpha-beta search. Thus, investment in move ordering quality (MVV-LVA, Killer Moves, and History Heuristics) directly enhances the safety of aggressive pruning heuristics.

7.3 Heuristic Marginal Utility and Search Failures

Analyzing individual heuristics reveals a stark divide between search-space reduction and playing-strength outcomes:

- **LMR Dominance vs. History Heuristics Marginal Utility:** Late Move Reduction (LMR) acts as the primary driver of node savings in Kronos, compressing the search tree geometrically by searching late-ranked moves at a reduced depth. However, its effectiveness is highly dependent on move sorting quality. In contrast, the History Heuristic exhibits very marginal utility. In a shallow search horizon (depth 3 to 4), the search tree is too small for quiet move beta cutoffs to accumulate statistically representative history scores. Consequently, the history table contains noisy weights that fail to improve move ordering, indicating that history-based sorting requires deeper search horizons to provide positive utility.
- **Null Move Pruning (NMP) Limitations and Pruning Failures:** At shallow horizons (such as depth 4), NMP fails to provide meaningful node reductions. Under our configuration ($R = 2$), a null-move search at depth 4 is executed at depth $d - 1 - R = 1$ ply. A 1-ply search with a null move incurs significant move verification and evaluation overhead, yet rarely triggers beta cutoffs because static positional evaluations at such shallow depths do not fluctuate enough to justify the pruning assumption. This shallow-depth pruning failure is well-documented in computer chess literature [7]: NMP with a constant reduction factor $R = 2$ is widely understood to be practically ineffective below depth 5. Consequently, the final engine configuration activates NMP only for depths $d \geq 5$.
- **SPRT Inconclusiveness and Test Resolution Bounds:** Under the configured SPRT protocol ($H_0 : \text{Elo} \leq 0$ vs. $H_1 : \text{Elo} \geq 10$), only quiescence search crossed the Wald boundary to be accepted. The remaining enhancements (PVS, LMR, NMP, and Killer/History) remained inconclusive. This indicates that while these heuristics succeed in reducing node count and executing searches faster, their individual playing strength contributions are below the statistical resolution of the test (< 10 Elo) under the configured sample size. To resolve such minor Elo differentials, a significantly larger sample size (exceeding thousands of games) would be required.
- **Single-Threaded Latency Constraints and V8 JIT Warming:** The performance regression observed at depth 7 is a consequence of single-threaded latency constraints combined with V8 execution mechanics. The V8 engine relies on hotness thresholds (number of function invocations) to trigger JIT compilation (from the Ignition interpreter up to Sparkplug and eventually TurboFan). In the initial stages of a search, code runs interpreted or baseline-compiled. As the search deepens, parallel JIT compilation threads optimize the hot paths, but the synchronization overhead and concurrent compilation tasks introduce transient latency spikes (JIT compilation stutter). At depth 7, these latency spikes cause the synchronous worker thread to exceed the tournament time control limit. The search must abort and fall back to the results of the last fully completed iterative deepening ply (depth 6). This hard latency ceiling indicates that single-threaded JavaScript cannot reliably execute deep game-tree searches under standard competitive time controls.

7.4 Latency Constraints and the Search Ceiling

Depth-scaling metrics exhibit consistent strength gains up to depth 6, followed by regression at depth 7. This is a latency-induced limitation of the single-threaded execution model. Addition-

ally, the non-monotonic NPS behavior remains an observed telemetry behavior requiring future investigation; we hypothesize this stems from transposition table cache collision dynamics and JIT recompilation triggers under V8.

At depth 7, the average search latency per position increases substantially, which frequently exceeds the tournament time limit. When the search is aborted mid-ply, the engine reverts to the best move from the last completed iterative deepening step (depth 6). In positions where depth-6 and depth-7 evaluations differ, the engine plays sub-optimally. This indicates a hard throughput ceiling, verifying that single-threaded JavaScript workers cannot reliably sustain depth-7 search under standard time controls. Pushing past this limit would require shared-memory parallel search strategies, such as Lazy SMP, operating over browser `SharedArrayBuffer` allocations.

7.5 Deterministic Seeding and SPRT Validation

To ensure reproducibility in playing strength evaluation, Kronos incorporates deterministic seeding and sequential testing.

First, the tournament runner shuffles the opening library using a custom LCG PRNG initialized with a fixed seed. This ensures that independent benchmark runs execute identical game sequences. Second, paired, color-flipped matches isolate playing strength from opening advantages. Each opening FEN is played twice, swapping color assignments between variants, which mitigates White's first-move advantage.

Third, Sequential Probability Ratio Testing (SPRT) provides a mathematically grounded termination criterion. Rather than evaluating a fixed game count, SPRT tracks the LLR against Wald boundaries, terminating as soon as the outcomes statistically confirm or reject a rating hypothesis. In practice, this sequential approach reduces the required game sample size by up to 40% compared to fixed-sample testing, making laptop-based systems evaluations viable.

Determinism and statistical validity: Because the search core is strictly deterministic (no temperature-scaled randomness or stochastic move selection), two identical engine variants playing from the same opening FEN will produce the exact same move sequence every time. The effective number of independent game outcomes per matchup is therefore bounded by the opening book size: $2 \times |\text{opening book}| = 100$ independent observations (50 positions $\times$ 2 color assignments). In cross-variant matchups (e.g., Alpha-Beta vs. Minimax), the two engine variants produce different move sequences from the same opening, so each of the 100 starting configurations yields a unique game trajectory. However, once all 100 configurations have been played, subsequent games are exact replays of earlier outcomes. Consequently, the reported SPRT game counts in Table 6.6 (ranging from 153 to 400 games) exceed this independence bound, meaning the effective sample sizes are capped at 100 regardless of the total games executed. The reported LLR values and confidence intervals are therefore computed over a sample containing duplicated outcomes, which inflates the apparent statistical power. We acknowledge this as a structural limitation of deterministic self-play: the SPRT termination decisions should be interpreted with the understanding that the true effective sample size is at most 100, and the reported confidence intervals understate the actual uncertainty. Crucially, these higher game counts (up to 400 games) were intentionally sustained to evaluate the long-running memory stability and garbage collection resilience of the tournament scheduler and worker threads over hundreds of consecutive games, rather than to claim additional statistical degrees of freedom.

To assess the impact of this sample-size inflation on our empirical findings, we analytically rescale the reported LLR metrics to the $N_{\text{eff}} = 100$ limit (effectively dividing the LLR by the inflation factor $N_{\text{total}}/100$). For Quiescence Search (EXP-6), scaling the LLR from 400 down to 100

games yields an adjusted LLR of $75.433/4 \approx 18.86$. Because this adjusted value remains significantly above the upper Wald boundary of 2.944, the acceptance of the playing-strength hypothesis for quiescence search remains statistically valid. For all other configurations (e.g., Move Ordering, EXP-2, where the LLR scales from 0.859 to ≈ 0.46; Iterative Deepening, EXP-5, where the LLR scales from -0.171 to ≈ -0.11), the adjusted metrics remain well within the Wald boundaries, confirming their inconclusive status. Therefore, while sample-size correction doubles the width of our rating confidence intervals, it does not alter the final binary acceptance or rejection decisions of the SPRT framework.

7.6 Draw-Rate Analysis in Self-Play Pools

During self-play matchups between variants of similar strength, draw rates reached 66.7% to 70.0% (Table 6.7). We identify four contributing factors to this high draw rate:

1. **Shallow Search Depth:** At depths 3 and 4, the engines cannot formulate long-term positional plans to create structural imbalances, resulting in defensive simplifications.

2. **Search Determinism:** Root move selection is entirely deterministic without randomization. In a closed self-play pool, this lacks variation, leading games into identical drawn lines. Combined with the bounded opening book, this determinism means that high draw rates between similarly-ranked variants are structurally inevitable rather than indicative of true playing strength parity.

3. **Move Limit Termination:** Even matchups frequently simplify into drawish endgames that are terminated by the runner's 200-ply limit.

4. **Repetition Detection Limitations:** The transposition table does not penalize move repetitions during search traversal. Consequently, the search core is prone to repeating moves when it evaluates the position as equal.

This high draw rate reduces the statistical information generated per game, expanding standard error margins and requiring larger game sample sizes to resolve minor Elo differences. In practice, this directly impacted the execution time of the automated tournament scheduler: because draws provide less LLR divergence than decisive results, matchups between variants of similar strength (such as EXP-3 and EXP-4) were forced to run close to or up to their maximum game limits before the LLR could cross a Wald boundary or stabilize within confidence limits.

7.7 Managed Runtime Engineering Lessons

Three design guidelines emerge from our study:

1. **Prioritize Allocation Elimination:** Replacing object allocations with in-place board state mutations yields a greater practical speedup than algorithmic modifications by removing GC spikes.

2. **Warm the JIT Compiler:** V8 JIT optimizations require several warm-up iterations to stabilize. Benchmarking timings recorded during the first few matches will result in distorted, high-latency metrics.

3. **Isolate Concurrent Tasks:** Without background Web Worker isolation, search execution blocks the browser main thread, freezing the user interface. Decoupling the search plane is a functional requirement for interactive dynamic runtimes.

Chapter 8

Limitations and Threats to Validity

This chapter details the throughput limitations of the Kronos search core and outlines potential threats to the validity of our experimental findings, alongside mitigation steps.

8.1 System Limitations

The primary limitation of Kronos is its search throughput, which is constrained by two architectural factors. First, the engine delegates board representation and move generation to the third-party `chess.js` library, which relies on a mailbox (0x88) board representation with complex string parsing and object validation on internal code paths. Unlike native C/C++ engines that utilize 64-bit integer bitboards updated via bitwise CPU operations, `chess.js` operates through object-based wrappers. Replacing `chess.js` with a custom typed-array bitboard generator would be the most impactful single optimization for throughput. Second, the Zobrist hashing subsystem uses JavaScript `BigInt` primitives for 64-bit key operations. While `BigInt` correctly preserves 64-bit precision, it incurs measurable overhead compared to the dual-32-bit integer representation commonly used in production JavaScript chess engines (e.g., representing a 64-bit hash as two 32-bit integers with native bitwise XOR operations). Together, these factors limit throughput to approximately 5,000 NPS (Table 6.4).

Consequently, the engine cannot search beyond 6 plies under standard time controls. Furthermore, the search core is single-threaded, preventing it from utilizing multi-core processor architectures.

8.2 Threats to Validity

8.2.1 Internal Validity

Internal validity threats concern execution variables that can affect timing and search statistics. In managed runtime environments, background operating system activity, JIT compiler warming, and garbage collection cycles introduce timing variations that can skew search latency benchmarks.

To mitigate these timing variations, all tournaments were executed on isolated hardware (Intel Core i5-1235U CPU, 16 GB RAM) with background processes disabled. JIT warming was managed by executing initial dummy games to ensure V8 compiler optimization completed before recording metrics. However, our benchmarks are inherently tied to this hardware profile and the Node.js v20.11.0 runtime (Google V8 Engine). Variations in clock speeds, memory latency, and JIT optimization heuristics across other platforms (such as Bun, Deno, or older V8 releases) represent

unmeasured threats to internal validity.

8.2.2 External Validity

External threats limit the generalizability of our findings. First, the engine's relative rating scaling and pruning efficiencies were evaluated using a specific opening library (`positions.json`) limited to 50 FEN positions. Results could differ under rare, highly unbalanced, or theoretically complex opening lines. We addressed color and opening bias by enforcing paired color-swapped matches, but the restricted size of the openings book remains a limitation.

Second, Kronos lacks several components common in modern tournament chess engines, such as endgame tablebases (e.g., Syzygy tablebases) and Neural Network evaluations (NNUE). The absence of tablebases increases draw rates in simplified endgames, as the engine cannot resolve forced mates or draws instantly. The absence of NNUE limits the positional and tactical accuracy of evaluations, restricting the engine's play to classical heuristic weightings (Piece-Square Tables).

8.2.3 Construct and Conclusion Validity

Construct validity threats relate to our playing strength metrics. The internal playing strength ratings reported in this work represent relative measurements within a closed self-play pool of Kronos engine variants, not absolute FIDE-style ratings. To provide an external reference, we benchmarked Kronos against depth-limited Stockfish 18 configurations. Although these Stockfish calibration matches provide a preliminary external reference anchored against fixed-depth configurations, depth-limited Stockfish does not degrade linearly: its NNUE evaluation architecture is designed for deep search propagation, and restricting Stockfish to depths 1–3 non-linearly cripples its evaluation accuracy. These matchup differentials therefore cannot be directly mapped to standard FIDE or CCRL Elo scales, and projected ratings for deeper Stockfish configurations are speculative estimates that should not be treated as measured empirical data.

Additionally, statistical conclusion validity is bounded by the deterministic nature of the search core. Because Kronos uses strictly deterministic root move selection (no temperature scaling or stochastic sampling), the effective degrees of freedom in any matchup are bounded by the opening book size: $2 \times |\text{opening book}| = 100$ independent game outcomes per variant pair. Sample sizes beyond this bound do not contribute new statistical information.

Finally, the 100 to 400 game tournaments executed at depth 4 provide standard error margins of approximately ± 17 to ± 34 Elo. However, the draw rates observed during self-play matches between similar variants (exceeding 66.7% to 70.0%) significantly reduce the statistical information gain per game, widening the confidence intervals. While this is sufficient to confirm significance for major structural changes (like quiescence search or move ordering), smaller rating differentials require much larger game samples to resolve conclusively. We address this limitation by reporting the full LLR status and trinomial confidence intervals for all tournament outcomes, explicitly identifying cases where results remain inconclusive.

8.2.4 Dependency Fragility and Reproducibility

A critical engineering threat to long-term reproducibility is Kronos's reliance on the internal, private API of the third-party `chess.js` library. To eliminate garbage collection overhead in the move generation loop, the custom wrapper class `ChessInstanceClone` bypasses the library's public interfaces and directly invokes private methods (e.g., `_moves()`, `_makeMove()`, and `_undoMove()`). While this achieves GC neutrality in the current workspace (Node.js environment with `chess.js` v1.4.0), it introduces severe dependency fragility. Future minor or major updates to `chess.js` that alter

or remove these internal private functions will break the search core. To mitigate this risk, future work must transition from third-party wrappers to an independent, custom-built move generator.

Chapter 9

Future Work

Several opportunities exist to extend the performance and capabilities of Kronos:

- **Custom Move Generator and Board Representation:** The current engine delegates move generation to the third-party `chess.js` library, which is the dominant throughput bottleneck. Replacing `chess.js` with a custom typed-array bitboard generator would eliminate the mailbox overhead and is expected to increase NPS by an order of magnitude, even within JavaScript.
- **Dual-32-bit Zobrist Hashing:** The current Zobrist hashing implementation uses JavaScript `BigInt` primitives for 64-bit keys. A standard optimization in JavaScript chess engines is to represent each 64-bit hash as two 32-bit integers, enabling native bitwise XOR operations without `BigInt` object overhead. This migration would reduce per-node hashing cost on the hot search path.
- **WebAssembly Migration:** Migrating the core search loop and board state mutations to WebAssembly (Wasm) represents a natural next step. While V8 JIT compilation offers optimized execution speed, WebAssembly would eliminate JIT warm-up times and compile-timing variations, providing consistent execution speeds.
- **Neural Network Evaluations (NNUE):** Replacing the classical static evaluation function with an Efficiently Updatable Neural Network (NNUE) would significantly improve evaluation quality. Future work will explore implementing NNUE inference inside web runtimes, utilizing WebGL/WebGPU or Wasm vector instructions.
- **Parallel Search Implementations:** The current Kronos worker architecture is single-threaded. We plan to explore parallel search algorithms, such as Lazy SMP or Principal Variation Split, utilizing shared memory abstractions (`SharedArrayBuffer`) and multi-worker execution pools to scale search performance on multi-core systems.
- **Caching, Tablebases, and Variance Heuristics:** Integrating endgame tablebase caching (such as Syzygy tablebases) and expanding our opening books would improve playing strength in the opening and endgame phases. Additionally, to mitigate the high draw rates (66.7%–70.0%) observed in self-play matches between similar variants, we plan to explore introducing temperature-scaled randomness or multi-PV (Principal Variation) selections in the opening plies to force more variance and reduce these deterministic draw loops.

Chapter 10

Conclusion

We have presented Kronos, a modular chess engine and automated benchmarking suite implemented in Node.js and JavaScript. Under V8 runtime constraints, we systematically evaluated search heuristics to measure their contribution to search space compression and playing strength. The results suggest that managed runtimes can execute classical search loops stably and efficiently when heap allocations are eliminated on the hot path.

Our empirical findings indicate a clear hierarchy of heuristic impact. Late Move Reduction is the most significant search reduction optimization, contributing a 66.3% node reduction. Quiescence search is critical, as its deactivation results in a 371 Elo rating regression—the largest performance drop in our ablation study. Move ordering serves as the foundation on which both LMR and PVS depend; without it, pruning heuristics lose their efficiency. Together, the full heuristic stack compresses the search space by 82.4% at depth 4, while reaching a peak relative self-play rating of 1,400 Elo at depth 6.

The depth-7 regression to 1,365 Elo highlights a latency ceiling imposed by single-threaded execution. When the search latency exceeds the tournament time limit, the engine falls back to depth-6 evaluations in tactically sharp positions. Overcoming this ceiling requires parallel search, which remains a primary avenue for future work.

10.1 Kronos as a Reproducible Research Framework

Beyond the immediate experimental outcomes, the primary contribution of Kronos is the framework itself. Any researcher with a Node.js environment can clone the repository, run the benchmarking pipeline, and reproduce the complete tournament workflow—from opening shuffle through SPRT verification to report generation—with a single command. All seeds, configuration flags, and opening libraries are version-controlled.

This accessibility distinguishes Kronos from prior JavaScript chess implementations, which were primarily interactive games without benchmarking infrastructure. Kronos provides a platform for investigating questions beyond this report, including the execution behavior of neural network evaluation functions on managed runtimes, or parallel search scaling via SharedArrayBuffer.

10.2 Reproducibility Reference

The Kronos source code, benchmark scripts, opening libraries, and result datasets are organized as follows:

- `src/engine/` — Search core, evaluation, transposition table, and move ordering.
- `benchmark/pipeline/` — Tournament runner, SPRT evaluator, and report generator.
- `benchmark/openings/` — Version-controlled opening FEN library.
- `benchmark/seeds/` — Fixed LCG seed values for each experiment.
- `benchmark/output/` — Raw PGN game files and report artifacts.
- `research-paper/` — This LaTeX manuscript, figures, and tables.

To reproduce the experimental suite, execute `npm run benchmark` from the project root. Individual experiments can be executed by specifying a configuration file:

```
node benchmark/pipeline/pipelineManager.js --config configs/exp-lmr.json
```

Figure generation from PGN output is automated via `benchmark/reports/exportOrdo.js`. All results reported in this paper are linked to specific seed values in the `benchmark/seeds/` directory and can be reproduced on any platform running Node.js ≥ 18.

This combination of a reproducible pipeline, statistically validated results, and open-source architecture presents Kronos as an accessible research platform.

Bibliography

[1] G. Richards, S. Lebresne, B. Burg, and J. Vitek, "An analysis of the dynamic behavior of javascript programs," in *Proceedings of the 31st ACM PLAN Conference on Programming Language Design and Implementation*, 2010, pp. 1–12.

[2] M. Selaković and M. Pradel, "Performance issues and optimizations in javascript: An empirical study," in *Proceedings of the International Conference on Software Engineering*, 2016, pp. 74–85.

[3] R. Jones, A. Hosking, and E. Moss, *The Garbage Collection Handbook: The Art of Automatic Memory Management*. CRC Press, 2011.

[4] T. Hunter II and L. Bryan, *Multithreaded JavaScript: Concurrency in Client-Side Writing*. O'Reilly Media, 2021.

[5] D. E. Knuth and R. W. Moore, "An analysis of alpha-beta pruning," *Artificial Intelligence*, vol. 6, no. 4, pp. 293–326, 1975.

[6] T. A. Marsland and M. S. Campbell, "Parallel search of game trees," *Computer Physics Communications*, vol. 26, no. 3-4, pp. 307–315, 1982.

[7] C. Donninger, "Null-move search in computer chess," *ICCA Journal*, vol. 16, no. 1, pp. 3–9, 1993.

[8] J. Schaeffer, "The history heuristic and alpha-beta search enhancements," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 11, no. 11, pp. 1203–1212, 1989.

[9] A. L. Zobrist, "A new hashing method with applications for game-playing," Computer Sciences Department, University of Wisconsin-Madison, Tech. Rep. 88, 1970.

[10] C. E. Shannon, "Programming a computer for playing chess," *Philosophical Magazine*, vol. 41, no. 314, pp. 256–275, 1950.

Appendix A

Engine Configuration Parameters

This appendix contains sample configuration schemas, opening FEN libraries, Stockfish calibration models, and raw runtime logs.

A.1 Search Engine Configuration Schema

The engine variant behaviors are defined using a JSON configuration file. Below is the configuration spec utilized by the tournament runner to initialize each engine instance:

```
{
  "search": {
    "useAlphaBeta": true,
    "useIterativeDeepening": true,
    "useMoveOrdering": true,
    "useTranspositionTable": true,
    "useQuiescence": true,
    "usePVS": true,
    "useLMR": true,
    "useNMP": true
  },
  "evaluation": {
    "usePieceSquareTables": true,
    "weights": {
      "pawn": 100,
      "knight": 320,
      "bishop": 330,
      "rook": 500,
      "queen": 900,
      "king": 20000
    },
    "taperedEndgameThreshold": 1600
  },
  "cache": {
    "maxTranspositionTableSize": 500000
  }
}
```

A.2 Opening Book Sample FEN Seeds

To evaluate playing strength without bias, games are seeded with positions from a randomized opening library. Below is a selection of the baseline opening FENs used:

1. `rnbqkbnr/pppppppp/8/8/4P3/8/PPPP1PPP/RNBQKBNR b KQkq e3 0 1` (King's Pawn Opening)
2. `rnbqkbnr/pppppppp/8/8/3P4/8/PPP1PPP/RNBQKBNR b KQkq d3 0 1` (Queen's Pawn Opening)
3. `rnbqkbnr/pppp1ppp/8/4p3/4P3/8/PPPP1PPP/RNBQKBNR w KQkq e6 0 2` (Open Game)
4. `r1bqkbnr/pppp1ppp/2n5/4p3/4P3/5N2/PPPP1PPP/RNBQKB1R w KQkq - 2 3` (King's Knight Opening)
5. `r1bqkbnr/pppp1ppp/2n5/1B2p3/4P3/5N2/PPPP1PPP/RNBQK2R b KQkq - 3 3` (Ruy Lopez)

Appendix B

Stockfish Calibration Notes

Extrapolated playing strength projections for deeper Stockfish configurations (Depths 4–8) are intentionally omitted from this work. Engine strength scaling is strictly non-linear and subject to diminishing returns, making linear extrapolation from shallow-depth calibration data scientifically invalid. Furthermore, as noted in Section 6, depth-limited Stockfish's NNUE evaluation architecture degrades non-linearly, rendering even the measured shallow-depth matchup differentials unsuitable as anchors for extrapolation. Future work requiring deeper calibration should conduct direct head-to-head matches at each target depth.

Appendix C

Raw Telemetry Logs

We provide a sample telemetry trace recorded during calibration matches to document V8 runtime memory behavior:

```
[Kronos Profiler] Game 5 Complete.
[Memory Profile] RSS: 75.26 MB | Heap Total: 45.26 MB | Heap Used: 26.60 MB
[Cache Telemetry] Transposition Table Size: 48 entries | Purging.
[Kronos Profiler] Game 10 Complete.
[Memory Profile] RSS: 77.31 MB | Heap Total: 46.76 MB | Heap Used: 24.98 MB
[Cache Telemetry] Transposition Table Size: 43 entries | Purging.
```

Appendix D

Reproducibility Checklist

To ensure complete empirical transparency and support future verification studies, we provide a systems reproducibility checklist for the Kronos benchmarking suite.

Table D.1. Reproducibility Configuration Checklist. Item indicates the hardware or software parameter; Value / Source lists the specific version, hardware model, or source identifier used to execute the experiments.

Item	**Value / Source**
Code Repository	https://github.com/piyush2180/kronos
Commit Hash	`53389c710f974b8e`
Runtime Environment	Node.js v20.11.0 (Google V8 Engine)
Host Hardware (CPU)	Intel Core i5-1235U (10 Cores, 12 Threads)
Host Hardware (RAM)	16 GB RAM (16,384 MB)
Host OS	Windows 11 Home Single Language (Build 26100)
PRNG Determinism Seed	`42` (fixed inside LCG shuffle)
Opening Book Library	50 balanced positions (`openings.json`)
SPRT Error Bounds	$\alpha = 0.05$, $\beta = 0.05$
Fitted Bradley-Terry	Anchored to Baseline Minimax (v1.0) = 1000